

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

Exp. No: 4

Date: 31-07-2026

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Implementation of Desk Calculator using Yacc Tool

Aim

To write a Lex program that recognizes the tokens needed by a Yacc parser, and a Yacc desk calculator that evaluates arithmetic expressions using +, -, *, /, ^, (and), with the correct precedence and associativity for each operator.

Tools Used

Flex

Bison

GCC

Code (ex.1)

```
%{
#include "y.tab.h"
%}

%%

[0-9]+(\.[0-9]+)? { yylval.dval = atof(yytext); return NUMBER; }

[ \t]           ;

\n              return '\n';

[+ \- * / ^()]   return yytext[0];

.               ;

%%
```

```
int yywrap(void) {
```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

    return 1;
}

```

Code (ex.y)

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int yylex(void);
void yyerror(char *s);
%}

%union {
    double dval;
}

%token <dval> NUMBER
%type <dval> expr

%left '+' '-'
%left '*' '/'
%right '^'

%start program

%%

```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

program : program line

|
;

line : expr '\n' { printf("Result = %g\n", \$1); }

| '\n'
| error '\n' { yyerror; }
;

expr : expr '+' expr { \$\$ = \$1 + \$3; }

| expr '-' expr { \$\$ = \$1 - \$3; }
| expr '*' expr { \$\$ = \$1 * \$3; }
| expr '/' expr { \$\$ = \$1 / \$3; }
| expr '^' expr { \$\$ = pow(\$1, \$3); }
| '(' expr ')' { \$\$ = \$2; }
| NUMBER { \$\$ = \$1; }
;

%%

void yyerror(char *s) {

printf("%s\n", s);
}

int main(void) {

printf("Enter expression:\n");
yyparse();
return 0;

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

}

Sample Inputs (Test Cases)

3+9

3+9*6

(3+4)*7

(3-4)+(7*6)

5/7+2

4^2^1

(2^3)^2

Commands Used

lex ex.l

yacc -d ex.y

cc lex.yy.c y.tab.c -lfl -lm -o calc

./calc

Output:

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

kushaal@Kushaals-MacBook-Air Assignment 4 % yacc -d ex.y
kushaal@Kushaals-MacBook-Air Assignment 4 % gcc lex.yy.c y.tab.c -o calc -lm
kushaal@Kushaals-MacBook-Air Assignment 4 % ./calc
Enter expression:
2+3*4
Result = 14

5+2
Result = 7

9/3+2^4
Result = 19

```

Learning Outcomes

Understood how Yacc resolves ambiguity in an arithmetic grammar through precedence and associativity declarations (%left, %right).

Learned that the grammar needs a left-recursive start production such as program : program line | ; so the parser keeps reading lines instead of stopping after the first.

Learned how Lex and Yacc exchange tokens through a %union-typed yylval/YYSTYPE, and why that type has to be the same in the generated lexer and parser.

Used Yacc's error token and yyerror to recover from a bad line and keep the calculator running.